

4. System Analysis & Design

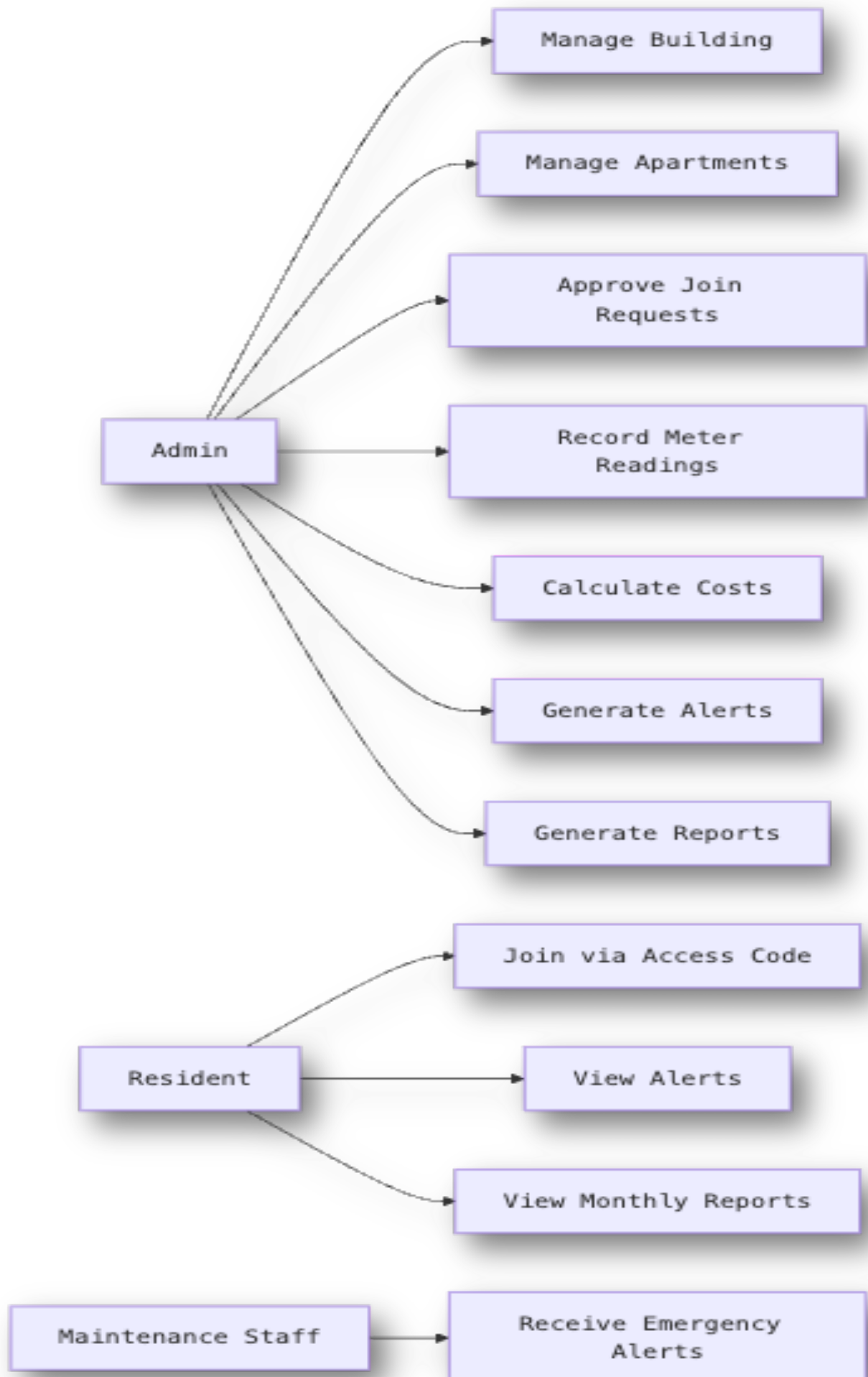
4.1 Problem statement and objectives

4.2 Use case diagram

- Actors:
- Admin
- Resident
- Maintenance Staff
- System

Use cases:

- Manage Building
- Manage Apartments
- Approve Join Requests
- Record Meter Readings
- Calculate Costs
- Generate Alerts
- Generate Reports
- View Reports



Admin

- Manage Building
- Manage Apartments
- Approve Join Requests
- Record Meter Readings
- Calculate Costs
- Generate Alerts
- Generate Reports
- Resident
- Join via Access Code
- View Alerts
- View Monthly Reports
- Maintenance Staff
- Receive Emergency Alerts

4.3 Software architecture

A suitable architecture is:

1. Flutter app for presentation layer
2. Supabase for authentication, backend services, and PostgreSQL database
3. SQLite for local cache/offline mode
4. Report generation module for monthly PDFs
5. Notification/alert module
6. Sync service for offline-first reconciliation

4.4 Architecture style

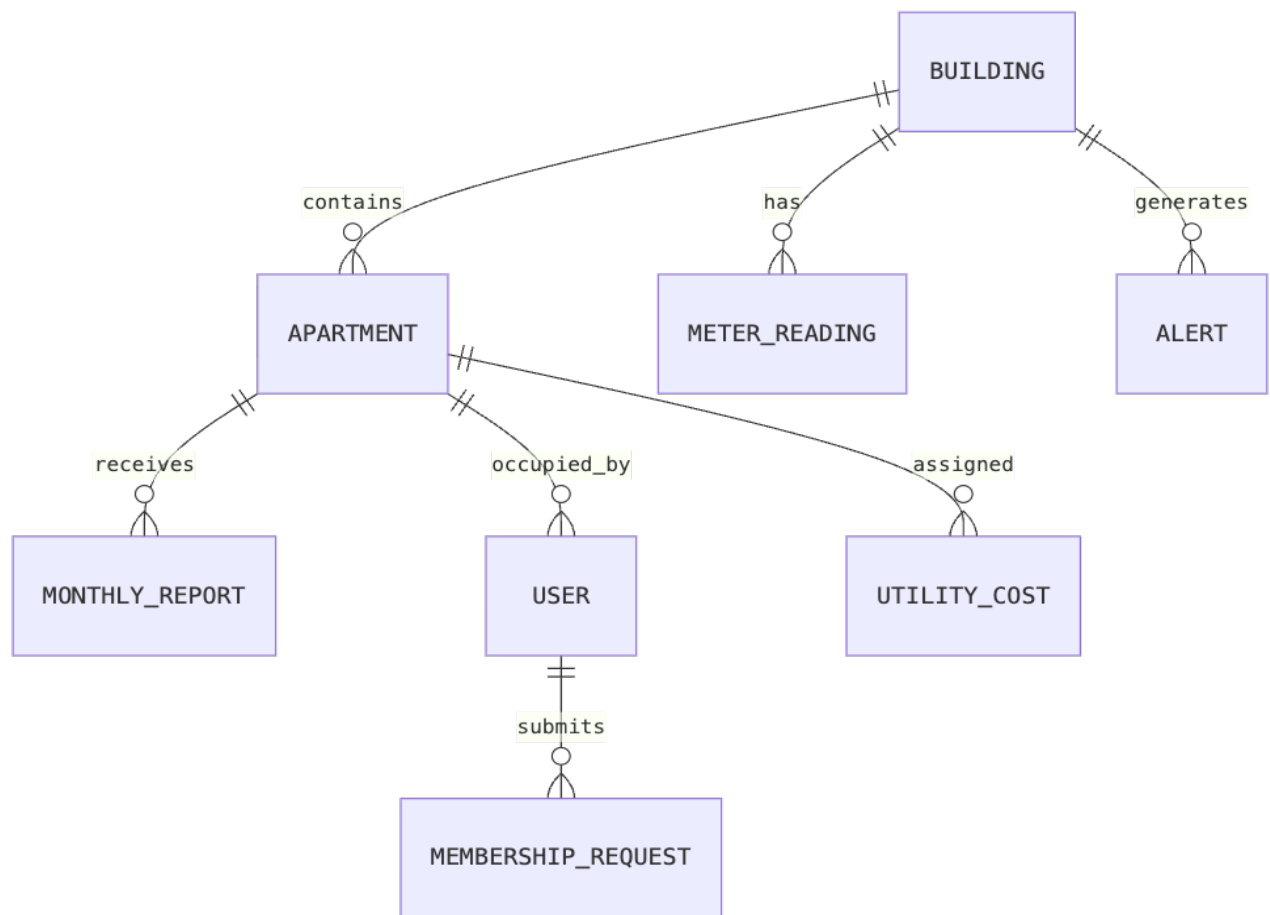
- Recommended:
- Client-server architecture
- Layered architecture
- Offline-first mobile architecture
- Possibly MVVM or Clean Architecture inside Flutter for maintainability

4.5 Database design and ERD

- Likely entities:
- Building
- Apartment
- User
- MembershipRequest
- MeterReading
- UtilityCost
- Alert
- MonthlyReport
- AdminApproval
- NotificationLog

Example high-level relationships:

- One building has many apartments
- One apartment has many residents
- One building has many meter readings
- One apartment has many monthly reports
- One building has many alerts
- Diagram:
- contains
- has
- generates
- receives
- submits
- occupied_by
- assigned
- BUILDING
- APARTMENT
- METER_READING
- ALERT
- MONTHLY_REPORT
- USER
- MEMBERSHIP_REQUEST
- UTILITY_COST



4.6 Logical schema

Example tables:

- buildings
- apartments
- users
- membership_requests
- meter_readings
- utility_distributions
- alerts
- monthly_reports
- audit_logs
- For each table, document:
 - attributes
 - primary key
 - foreign keys

- constraints
- indexes

4.7 Data flow diagram

- Context-level DFD:
- Resident/Admin provides input
- System processes membership, readings, billing, reports
- Database stores records
- Reports and alerts are output

4.8 Sequence diagrams

- Include at least:
- Resident joins via access code
- Admin records meter readings and generates monthly costs
- Monthly PDF report generation
- Offline sync flow

4.9 Activity diagrams

- Good candidates:
- Join request approval workflow
- Monthly billing/report generation workflow
- Emergency alert flow
- 4.10 State diagram
- You can use a state diagram for:
- Membership request: Pending → Approved / Rejected
- Alert: Created → Sent → Read / Unread → Archived
- Report: Draft → Generated → Published

4.11 Class diagram

- Important classes/modules:
- User
- Admin
- Resident
- Building
- Apartment
- MeterReading

- UtilityCalculator
- ReportService
- AlertService
- SyncManager